

Arquitectura en Capas (Layered Architecture)



Objetivos

- Entender la arquitectura en capas y sus beneficios
- Conocer las responsabilidades de cada capa
- Aplicar el principio de separación de responsabilidades
- Organizar un proyecto FastAPI en capas

¿Qué es la Arquitectura en Capas?

La **arquitectura en capas** organiza el código en grupos horizontales donde cada capa tiene una responsabilidad específica y solo puede comunicarse con las capas adyacentes.

Principios Fundamentales

1. **Separación de responsabilidades:** Cada capa tiene un propósito único
2. **Dependencia unidireccional:** Las capas superiores dependen de las inferiores, nunca al revés
3. **Abstracción:** Cada capa oculta su implementación interna

Las Tres Capas Principales

1. Presentation Layer (Capa de Presentación)

Responsabilidades:

- Recibir requests HTTP
- Validar datos de entrada (Pydantic)
- Serializar respuestas
- Manejar autenticación/autorización HTTP
- Documentación OpenAPI

```
# routers/products.py
from fastapi import APIRouter, Depends, HTTPException, status

from schemas.product import ProductCreate, ProductResponse
from services.product import ProductService
from dependencies import get_product_service

router = APIRouter(prefix="/products", tags=["products"])

@router.post("/", response_model=ProductResponse, status_code=201)
def create_product(
    data: ProductCreate, # Validación automática
    service: ProductService = Depends(get_product_service)
):
    """
    Crea un nuevo producto.

    Esta capa SOLO se encarga de:
    - Recibir el request
    - Validar datos (Pydantic)
    - Llamar al service
    - Retornar response
    """
    try:
        return service.create(data)
    except ProductAlreadyExistsError as e:
        raise HTTPException(status_code=409, detail=str(e))
```

2. Application Layer (Capa de Aplicación/Servicios)

Responsabilidades:

- Lógica de negocio
- Orquestación de operaciones
- Validaciones de negocio
- Transacciones (via Unit of Work)

```
# services/product.py
from schemas.product import ProductCreate, ProductUpdate
from repositories.product import ProductRepository
from models.product import Product

class ProductService:
    """
```

Capa de servicios: contiene la lógica de NEGOCIO.

NO conoce HTTP, solo trabaja con objetos de dominio.
"""

```
def __init__(self, repo: ProductRepository):
    self.repo = repo

def create(self, data: ProductCreate) -> Product:
    """
    Crea un producto aplicando reglas de negocio.
    """
    # Regla de negocio: SKU debe ser único
    if self.repo.exists_by_sku(data.sku):
        raise ProductAlreadyExistsError(f"SKU '{data.sku}' already exists")

    # Regla de negocio: precio mínimo
    if data.price < 0.01:
        raise InvalidPriceError("Price must be at least 0.01")

    # Crear entidad
    product = Product(
        name=data.name,
        sku=data.sku,
        price=data.price,
        stock=data.stock
    )

    return self.repo.add(product)
```

3. Data Access Layer (Capa de Acceso a Datos)

Responsabilidades:

- Persistencia de datos
- Queries a la base de datos
- Mapeo objeto-relacional
- Transacciones de bajo nivel

```
# repositories/product.py
from typing import TypeVar, Generic
from sqlalchemy import select
from sqlalchemy.orm import Session

from models.product import Product

class ProductRepository:
    """
    Capa de datos: acceso a la base de datos.

    NO conoce reglas de negocio, solo CRUD y queries.
    """

    def __init__(self, db: Session):
        self.db = db
```

```

def add(self, product: Product) -> Product:
    """Persiste un producto"""
    self.db.add(product)
    self.db.flush()
    self.db.refresh(product)
    return product

def get_by_id(self, product_id: int) -> Product | None:
    """Obtiene producto por ID"""
    return self.db.get(Product, product_id)

def exists_by_sku(self, sku: str) -> bool:
    """Verifica si existe un producto con ese SKU"""
    stmt = select(Product).where(Product.sku == sku)
    return self.db.execute(stmt).scalar_one_or_none() is not None

```

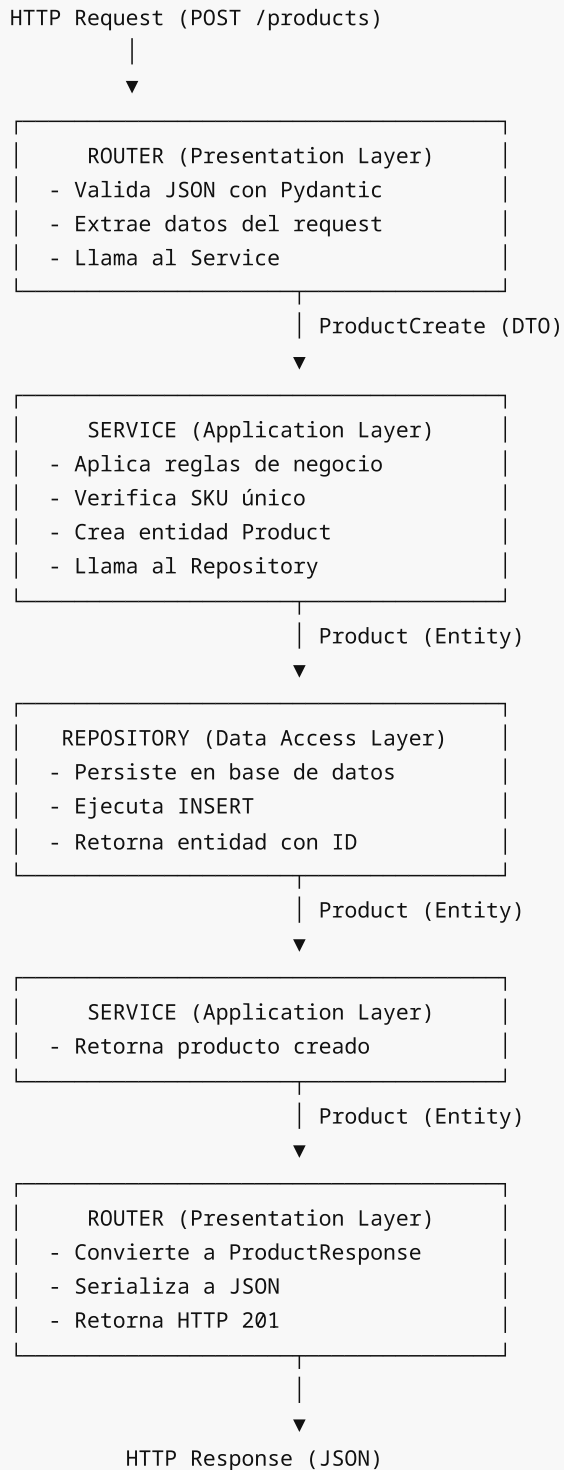
Estructura de Proyecto

```

src/
├─ main.py                # Punto de entrada FastAPI
├─ config.py              # Configuración
├─ database.py            # Conexión DB
├─
├─ models/                # 🗃️ DATA LAYER - Entidades SQLAlchemy
│   ├── __init__.py
│   ├── base.py
│   ├── product.py
│   └── category.py
├─
├─ repositories/          # 🗃️ DATA LAYER - Acceso a datos
│   ├── __init__.py
│   ├── base.py
│   ├── product.py
│   └── category.py
├─
├─ schemas/               # 📄 PRESENTATION - DTOs Pydantic
│   ├── __init__.py
│   ├── product.py
│   └── category.py
├─
├─ services/              # ⚙️ APPLICATION - Lógica de negocio
│   ├── __init__.py
│   ├── product.py
│   └── category.py
├─
├─ routers/               # 🌐 PRESENTATION - Endpoints HTTP
│   ├── __init__.py
│   ├── products.py
│   └── categories.py
├─
├─ exceptions/            # ❌ Excepciones personalizadas
│   ├── __init__.py
│   ├── base.py
│   └── product.py
├─

```

🔄 Flujo de una Request



✅ Beneficios de la Arquitectura en Capas

Beneficio	Descripción
Mantenibilidad	Cambios aislados en una capa

Testabilidad	Cada capa se puede testear independientemente
Reusabilidad	Services reutilizables en diferentes contextos
Escalabilidad	Equipos pueden trabajar en paralelo
Comprensión	Estructura predecible y familiar

⚠ Errores Comunes

✗ Lógica de negocio en Router

```
# ✗ MAL - lógica en router
@router.post("/products/")
def create_product(data: ProductCreate, db: Session = Depends(get_db)):
    # Esto debería estar en el Service
    if db.query(Product).filter(Product.sku == data.sku).first():
        raise HTTPException(status_code=409, detail="SKU exists")

    product = Product(**data.model_dump())
    db.add(product)
    db.commit()
    return product
```

✓ Lógica en Service

```
# ✓ BIEN - router delega al service
@router.post("/products/")
def create_product(
    data: ProductCreate,
    service: ProductService = Depends(get_product_service)
):
    try:
        return service.create(data)
    except ProductAlreadyExistsError as e:
        raise HTTPException(status_code=409, detail=str(e))
```

📚 Recursos

- [Clean Architecture - Robert C. Martin](#)
- [Layered Architecture - Martin Fowler](#)

✓ Checklist

- ☐ Entiendo las tres capas principales
- ☐ Sé qué responsabilidades tiene cada capa
- ☐ Puedo organizar un proyecto en capas
- ☐ Entiendo el flujo de datos entre capas